

# TOPSIS Method - Smartphone Selection Report

Enis Ögdüm

## 1. Problem Description

**Objective:** Choose the best smartphone from 10 modern alternatives. **Method:** TOPSIS (Technique for Order Preference by Similarity to Ideal Solution) — implemented from scratch using Python and numpy without any external TOPSIS libraries.

### Criteria & Weights

- **Price (USD)** — weight 0.35 — **MIN** (lower price is better)
- **Battery (mAh)** — weight 0.15 — **MAX** (higher capacity is better)
- **Camera Score** — weight 0.25 — **MAX** (DxOMark-equivalent score)
- **Performance Score** — weight 0.25 — **MAX** (Geekbench 6 Multi-Core approx.)

### Data Sources

- **Prices:** Official launch prices / GSMArena
- **Battery:** Manufacturer specifications / GSMArena
- **Camera Score:** DxOMark / community benchmarks
- **Performance:** Geekbench 6 Multi-Core (approximate)

## 2. Full TOPSIS Procedure & Steps

Below is the step-by-step computational procedure as executed by the custom script mapping the entire TOPSIS logic:

| =====                              |             |               |              |
|------------------------------------|-------------|---------------|--------------|
| STEP 0 – Initial Decision Matrix X |             |               |              |
| =====                              |             |               |              |
| Alternative                        | Price (USD) | Battery (mAh) | Camera Score |
| Perf. Score                        |             |               |              |
| -----                              |             |               |              |
| iPhone 15 Pro                      | 999.0000    | 3274.0000     | 154.0000     |
| 7192.0000                          |             |               |              |
| Samsung Galaxy S24 Ultra           | 1299.0000   | 5000.0000     | 144.0000     |
| 7249.0000                          |             |               |              |
| Google Pixel 8 Pro                 | 999.0000    | 5050.0000     | 153.0000     |
| 4425.0000                          |             |               |              |
| OnePlus 12                         | 799.0000    | 5400.0000     | 140.0000     |
| 6800.0000                          |             |               |              |
| Xiaomi 14 Pro                      | 899.0000    | 4880.0000     | 144.0000     |
| 6900.0000                          |             |               |              |
| Sony Xperia 1 V                    | 1199.0000   | 5000.0000     | 136.0000     |

|                      |          |           |          |
|----------------------|----------|-----------|----------|
| 5200.0000            |          |           |          |
| Asus Zenfone 10      | 699.0000 | 4300.0000 | 130.0000 |
| 5600.0000            |          |           |          |
| Motorola Edge 50 Pro | 650.0000 | 4500.0000 | 133.0000 |
| 4800.0000            |          |           |          |
| Nothing Phone (2)    | 599.0000 | 4700.0000 | 135.0000 |
| 4600.0000            |          |           |          |
| Honor Magic 6 Pro    | 999.0000 | 5600.0000 | 158.0000 |
| 7100.0000            |          |           |          |

| =====                                            |             |               |              |
|--------------------------------------------------|-------------|---------------|--------------|
| STEP 1 – Normalized Matrix R                     |             |               |              |
| $r_{ij} = x_{ij} / \sqrt{\sum_{k=1}^n x_{kj}^2}$ |             |               |              |
| =====                                            |             |               |              |
| Alternative                                      | Price (USD) | Battery (mAh) | Camera Score |
| Perf. Score                                      |             |               |              |
| -----                                            |             |               |              |
| iPhone 15 Pro                                    | 0.3361      | 0.2152        | 0.3406       |
| 0.3735                                           |             |               |              |
| Samsung Galaxy S24 Ultra                         | 0.4370      | 0.3287        | 0.3185       |
| 0.3765                                           |             |               |              |
| Google Pixel 8 Pro                               | 0.3361      | 0.3320        | 0.3384       |
| 0.2298                                           |             |               |              |
| OnePlus 12                                       | 0.2688      | 0.3550        | 0.3096       |
| 0.3532                                           |             |               |              |
| Xiaomi 14 Pro                                    | 0.3024      | 0.3208        | 0.3185       |
| 0.3584                                           |             |               |              |
| Sony Xperia 1 V                                  | 0.4034      | 0.3287        | 0.3008       |
| 0.2701                                           |             |               |              |
| Asus Zenfone 10                                  | 0.2352      | 0.2827        | 0.2875       |
| 0.2908                                           |             |               |              |
| Motorola Edge 50 Pro                             | 0.2187      | 0.2958        | 0.2941       |
| 0.2493                                           |             |               |              |
| Nothing Phone (2)                                | 0.2015      | 0.3090        | 0.2986       |
| 0.2389                                           |             |               |              |
| Honor Magic 6 Pro                                | 0.3361      | 0.3681        | 0.3494       |
| 0.3688                                           |             |               |              |

| =====                                         |             |               |              |
|-----------------------------------------------|-------------|---------------|--------------|
| STEP 2 – Weighted Normalized Matrix V = R × w |             |               |              |
| =====                                         |             |               |              |
| Alternative                                   | Price (USD) | Battery (mAh) | Camera Score |
| Perf. Score                                   |             |               |              |
| -----                                         |             |               |              |
| iPhone 15 Pro                                 | 0.1176      | 0.0323        | 0.0851       |
| 0.0934                                        |             |               |              |
| Samsung Galaxy S24 Ultra                      | 0.1530      | 0.0493        | 0.0796       |
| 0.0941                                        |             |               |              |

|                                |        |        |        |
|--------------------------------|--------|--------|--------|
| Google Pixel 8 Pro<br>0.0575   | 0.1176 | 0.0498 | 0.0846 |
| OnePlus 12<br>0.0883           | 0.0941 | 0.0532 | 0.0774 |
| Xiaomi 14 Pro<br>0.0896        | 0.1059 | 0.0481 | 0.0796 |
| Sony Xperia 1 V<br>0.0675      | 0.1412 | 0.0493 | 0.0752 |
| Asus Zenfone 10<br>0.0727      | 0.0823 | 0.0424 | 0.0719 |
| Motorola Edge 50 Pro<br>0.0623 | 0.0765 | 0.0444 | 0.0735 |
| Nothing Phone (2)<br>0.0597    | 0.0705 | 0.0463 | 0.0746 |
| Honor Magic 6 Pro<br>0.0922    | 0.1176 | 0.0552 | 0.0874 |

=====

STEP 3 – Ideal (A+) and Anti-Ideal (A-) Solutions

=====

| Solution<br>Perf. Score   | Price (USD) | Battery (mAh) | Camera Score |
|---------------------------|-------------|---------------|--------------|
| -----                     |             |               |              |
| Ideal (A+)<br>0.0941      | 0.0705      | 0.0552        | 0.0874       |
| Anti-Ideal (A-)<br>0.0575 | 0.1530      | 0.0323        | 0.0719       |

=====

STEP 4 – Euclidean Distances from A+ and A-

=====

| Alternative              | d+ (to Ideal) | d- (to Anti-Ideal) |
|--------------------------|---------------|--------------------|
| -----                    |               |                    |
| iPhone 15 Pro            | 0.0524        | 0.0521             |
| Samsung Galaxy S24 Ultra | 0.0830        | 0.0412             |
| Google Pixel 8 Pro       | 0.0600        | 0.0414             |
| OnePlus 12               | 0.0263        | 0.0699             |
| Xiaomi 14 Pro            | 0.0371        | 0.0597             |
| Sony Xperia 1 V          | 0.0767        | 0.0233             |
| Asus Zenfone 10          | 0.0316        | 0.0730             |
| Motorola Edge 50 Pro     | 0.0368        | 0.0775             |
| Nothing Phone (2)        | 0.0377        | 0.0837             |
| Honor Magic 6 Pro        | 0.0471        | 0.0567             |

=====

STEP 5 – Closeness Coefficient  $C_i$  & Final Ranking

$C_i = d^- / (d^+ + d^-)$  |  $C_i \in [0,1]$ , higher = better

=====

| Alternative | $C_i$ | Rank |
|-------------|-------|------|
|-------------|-------|------|

|                          |        |    |
|--------------------------|--------|----|
| iPhone 15 Pro            | 0.4984 | 7  |
| Samsung Galaxy S24 Ultra | 0.3315 | 9  |
| Google Pixel 8 Pro       | 0.4084 | 8  |
| OnePlus 12               | 0.7267 | 1  |
| Xiaomi 14 Pro            | 0.6165 | 5  |
| Sony Xperia 1 V          | 0.2326 | 10 |
| Asus Zenfone 10          | 0.6976 | 2  |
| Motorola Edge 50 Pro     | 0.6780 | 4  |
| Nothing Phone (2)        | 0.6893 | 3  |
| Honor Magic 6 Pro        | 0.5462 | 6  |

#### STEP 6 – Final Ranking (Best → Worst)

|                             |             |
|-----------------------------|-------------|
| 1. OnePlus 12               | Ci = 0.7267 |
| 2. Asus Zenfone 10          | Ci = 0.6976 |
| 3. Nothing Phone (2)        | Ci = 0.6893 |
| 4. Motorola Edge 50 Pro     | Ci = 0.6780 |
| 5. Xiaomi 14 Pro            | Ci = 0.6165 |
| 6. Honor Magic 6 Pro        | Ci = 0.5462 |
| 7. iPhone 15 Pro            | Ci = 0.4984 |
| 8. Google Pixel 8 Pro       | Ci = 0.4084 |
| 9. Samsung Galaxy S24 Ultra | Ci = 0.3315 |
| 10. Sony Xperia 1 V         | Ci = 0.2326 |

### 3. Results Interpretation

The TOPSIS method selected the **OnePlus 12** as the best overall choice out of the 10 smartphone alternatives.

Despite the heavy emphasis placed on price (weight = 0.35 / 35%), the OnePlus 12 managed to achieve rank #1 by delivering an incredibly strong balance across all four evaluation criteria. It offered a highly competitive price alongside exceptionally large battery capacity and high processing performance.

On the contrary, the **Sony Xperia 1 V** ranked last. This was mainly due to its high cost relative to its comparatively average battery and performance, which pushed it closest to the mathematically anti-ideal solution ( $A^-$ ) and geometrically farthest from our ideal solution ( $A^+$ ).

### 4. Source Code Implementation (topsis.py)

Below is the complete custom Python script providing the exact functionality and normalizations used in the calculations above:

```
import numpy as np
```

```
#
```

```

# TOPSIS Implementation
#
=====

def topsis(X, w, criteria_types):
    """
    TOPSIS Implementation from scratch.

    Args:
        X          : numpy array (n x m) – decision matrix
        w          : numpy array (m,)  – weights vector
        criteria_types : List of 'min'/'max' – one per criterion

    Returns:
        dict with all intermediate steps + final preference vector and
        ranking
    """
    n, m = X.shape

    # Step 1 – Vector Normalization
    #  $r_{ij} = x_{ij} / \sqrt{\sum_{k=1}^n x_{kj}^2}$ 
    norm_divisors = np.sqrt(np.sum(X ** 2, axis=0))
    R = X / norm_divisors

    # Step 2 – Weighted Normalized Matrix
    V = R * w

    # Step 3 – Ideal (A+) and Anti-Ideal (A-) Solutions
    ideal = np.zeros(m)
    anti_ideal = np.zeros(m)

    for j in range(m):
        if criteria_types[j] == 'max':
            ideal[j] = np.max(V[:, j])
            anti_ideal[j] = np.min(V[:, j])
        elif criteria_types[j] == 'min':
            ideal[j] = np.min(V[:, j])
            anti_ideal[j] = np.max(V[:, j])
        else:
            raise ValueError(f"Criterion type must be 'max' or 'min', got: {criteria_types[j]}")

    # Step 4 – Euclidean Distances
    d_plus = np.sqrt(np.sum((V - ideal) ** 2, axis=1))
    d_minus = np.sqrt(np.sum((V - anti_ideal) ** 2, axis=1))

    # Step 5 – Closeness Coefficient (Preference Vector)
    C = d_minus / (d_plus + d_minus)

```

```

# Step 6 – Ranking (higher C = better rank)
sorted_indices = np.argsort(C)[::-1]
rank_vector    = np.empty(n, dtype=int)
rank_vector[sorted_indices] = np.arange(1, n + 1)

return {
    'normalized_matrix' : R,
    'weighted_matrix'   : V,
    'ideal_solution'    : ideal,
    'anti_ideal_solution': anti_ideal,
    'distances_ideal'   : d_plus,
    'distances_anti_ideal': d_minus,
    'preference_vector' : C,
    'ranking_vector'    : rank_vector,
    'sorted_indices'    : sorted_indices,
}

#
=====
# Helper – Pretty-print a matrix as a table
#
=====

def print_matrix(data, row_labels, col_labels, title):
    """Print a 2-D array (or 1-D row) as a formatted table."""
    print(f"\n{'='*70}")
    print(f"  {title}")
    print(f"{'='*70}")

    col_w = 16
    row_w = 28

    # Header
    header = f"{'Alternative':<{row_w}}" + "".join(f"{'c':>{col_w}}" for c in
col_labels)
    print(header)
    print("-" * len(header))

    # Rows
    if np.ndim(data) == 1:
        data = data.reshape(1, -1)

    for i, label in enumerate(row_labels):
        row_str = f"{label:<{row_w}}" + "".join(f"{data[i, j]:>{col_w}.4f}"
for j in range(data.shape[1]))
        print(row_str)

```

```
def print_row(values, col_labels, label, row_w=28, col_w=16):
    row_str = f"{label:<{row_w}} " + "".join(f"{v:>{col_w}.4f}" for v in
values)
    print(row_str)
```

```
#
```

```
=====
# Decision Problem – Choosing a Smartphone
```

```
#
```

```
alternatives = [
    "iPhone 15 Pro",
    "Samsung Galaxy S24 Ultra",
    "Google Pixel 8 Pro",
    "OnePlus 12",
    "Xiaomi 14 Pro",
    "Sony Xperia 1 V",
    "Asus Zenfone 10",
    "Motorola Edge 50 Pro",
    "Nothing Phone (2)",
    "Honor Magic 6 Pro",
]
```

```
criteria      = ["Price (USD)", "Battery (mAh)", "Camera Score", "Perf.
Score"]
criteria_types = ['min', 'max', 'max', 'max']
w              = np.array([0.35, 0.15, 0.25, 0.25])
```

```
# Decision matrix [Price, Battery, Camera, Performance]
```

```
X = np.array([
    [ 999, 3274, 154, 7192], # iPhone 15 Pro
    [1299, 5000, 144, 7249], # Samsung Galaxy S24 Ultra
    [ 999, 5050, 153, 4425], # Google Pixel 8 Pro
    [ 799, 5400, 140, 6800], # OnePlus 12
    [ 899, 4880, 144, 6900], # Xiaomi 14 Pro
    [1199, 5000, 136, 5200], # Sony Xperia 1 V
    [ 699, 4300, 130, 5600], # Asus Zenfone 10
    [ 650, 4500, 133, 4800], # Motorola Edge 50 Pro
    [ 599, 4700, 135, 4600], # Nothing Phone (2)
    [ 999, 5600, 158, 7100], # Honor Magic 6 Pro
], dtype=float)
```

```
#
```

```
=====
# Run TOPSIS & Print Full Report
```

```
#
```

```

=====

if __name__ == "__main__":

    print("\n" + "#" * 70)
    print(" TOPSIS METHOD – Smartphone Selection")
    print(" Custom Python Implementation (no external TOPSIS library)")
    print("#" * 70)

    # — Problem description
    

---


    print(\"""
PROBLEM DESCRIPTION
-----
Objective : Choose the best smartphone from 10 modern alternatives.
Method    : TOPSIS (Technique for Order Preference by Similarity to
              Ideal Solution) – implemented from scratch using NumPy only.

CRITERIA & WEIGHTS
-----
Price (USD)      – weight 0.35 – MIN   (lower price is better)
Battery (mAh)    – weight 0.15 – MAX   (higher capacity is better)
Camera Score     – weight 0.25 – MAX   (DxOMark-equivalent score)
Performance Score – weight 0.25 – MAX   (Geekbench 6 Multi-Core approx.)

DATA SOURCES
-----
Prices           : Official launch prices / GSMArena
Battery          : Manufacturer specifications / GSMArena
Camera Score     : DxOMark / community benchmarks
Performance      : Geekbench 6 Multi-Core (approximate)
\\""")

    # — Step 0 – Initial Decision Matrix
    

---


    print_matrix(X, alternatives, criteria, "STEP 0 – Initial Decision Matrix
X")

    # — Run algorithm
    

---


    res = topsis(X, w, criteria_types)

    # — Step 1 – Normalised Matrix
    

---


    print_matrix(res['normalized_matrix'], alternatives, criteria,
                  "STEP 1 – Normalized Matrix  R\n"
                  "  r_ij = x_ij / sqrt( sum_{k=1}^{n} x_{kj}^2 )")

    # — Step 2 – Weighted Normalised Matrix

```

---

```

print_matrix(res['weighted_matrix'], alternatives, criteria,
             "STEP 2 – Weighted Normalized Matrix   $V = R \times w$ ")

# — Step 3 – Ideal & Anti-Ideal Solutions


---


print(f"\n{'='*70}")
print(" STEP 3 – Ideal (A+) and Anti-Ideal (A-) Solutions")
print(f"{'='*70}")
col_w, row_w = 16, 28
header = f"{'Solution':<{row_w}}" + "".join(f"{'c:>{col_w}}" for c in
criteria)
print(header)
print("-" * len(header))
print_row(res['ideal_solution'], criteria, "Ideal (A+)", row_w,
col_w)
print_row(res['anti_ideal_solution'], criteria, "Anti-Ideal (A-)",
row_w, col_w)

# — Step 4 – Distances


---


print(f"\n{'='*70}")
print(" STEP 4 – Euclidean Distances from A+ and A-")
print(f"{'='*70}")
dist_cols = ["d+ (to Ideal)", "d- (to Anti-Ideal)"]
dist_data = np.column_stack((res['distances_ideal'],
res['distances_anti_ideal']))
print_matrix(dist_data, alternatives, dist_cols, "Distances")

# — Step 5 – Closeness Coefficient & Ranking


---


print(f"\n{'='*70}")
print(" STEP 5 – Closeness Coefficient  $C_i$  & Final Ranking")
print(f"  $C_i = d- / (d+ + d-)$  |  $C_i \in [0,1]$ , higher = better")
print(f"{'='*70}")
print(f"\n {'Alternative':<28} {'Ci':>10} {'Rank':>6}")
print(" " + "-" * 46)
for i, alt in enumerate(alternatives):
    ci = res['preference_vector'][i]
    rank = res['ranking_vector'][i]
    print(f" {alt:<28} {ci:>10.4f} {rank:>6}")

# — Step 6 – Final Sorted Order


---


print(f"\n{'='*70}")
print(" STEP 6 – Final Ranking (Best → Worst)")
print(f"{'='*70}")
for position, idx in enumerate(res['sorted_indices'], start=1):
    alt = alternatives[idx]
```

```
ci = res['preference_vector'][idx]
print(f" {position:>2}. {alt:<28} Ci = {ci:.4f}")
```

#### # — Interpretation

---

```
best_idx = res['sorted_indices'][0]
worst_idx = res['sorted_indices'][-1]
best_alt = alternatives[best_idx]
worst_alt = alternatives[worst_idx]
best_ci = res['preference_vector'][best_idx]
worst_ci = res['preference_vector'][worst_idx]
```

```
print(f"\n\n"
{'='*70}
INTERPRETATION
{'='*70})
```

```
Best alternative : {best_alt} (Ci = {best_ci:.4f})
Worst alternative : {worst_alt} (Ci = {worst_ci:.4f})
```

The TOPSIS method selected the {best\_alt} as the best overall choice. Despite the heavy emphasis on price (weight = 0.35), it achieved rank #1 by delivering a strong balance across all four criteria – competitive pricing, solid battery, a good camera score, and high performance.

The {worst\_alt} ranked last mainly because its high price relative to its performance and battery pushed it closest to the anti-ideal solution and farthest from the ideal.

Note: Changing the weights will change the ranking. TOPSIS is sensitive to weight choices, which is a known limitation of the method.

```
{'='*70}
\n\n\n")
```